Architectural Patterns & System Design

Session 1 after second freeze

First ,Welcome Back !
;)

Table of contents

01

Why Architecture Exists

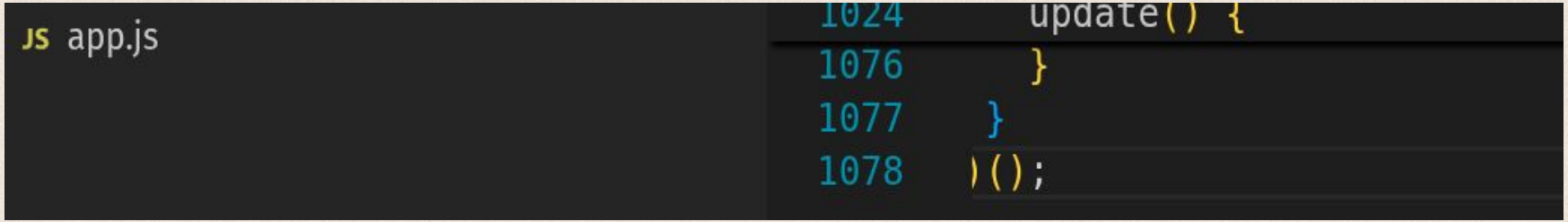
Architectural Patterns & System Design

- **Building software that survives growth.**

Today we're not learning Express or Nest.

We're learning how software is organized.

Imagine This...



Would you like to maintain this?

That's why Software Architecture exists.

What is Software Architecture?

Software Architecture

↓

How we organize our code.

What is Software Architecture?

Examples:

- Where does business logic live?
- Where does database logic live?
- Which modules should exist?
- How do modules communicate?

Architecture is the decisions that are expensive to change later.

Software Architecture

- **System architecture** is the high-level structure of an **entire system,** defining how software, hardware, databases, networks, external services, and infrastructure work together to meet business and technical requirements.

Then What is System Design?

System Design

Software Architecture

↓

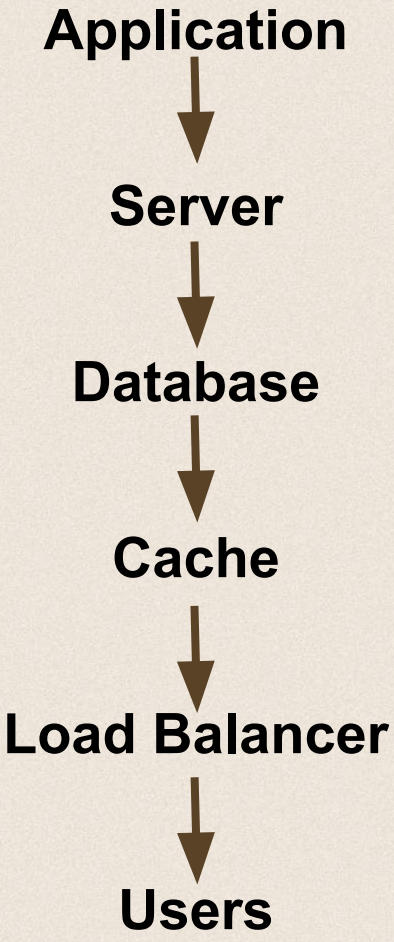
Inside application.

System Design

↓

Entire system.

System Design



System Design

- **System Design:** is the process of defining a system's architecture, components, interfaces, and data flow **to meet functional and non-functional requirements** , with the goal of building a system that can handle **scale**, **recover** from failures, and perform **reliably** under load.

System Architecture vs. Software Architecture

- The two overlap but aren't identical.
- **Software architecture** covers the **internal structure** of a software application: modules, layers, APIs, and runtime behavior.
- **System architecture** is broader. It includes the software, hardware, network, data storage, external systems, and humans in the loop.

Example: Online Shopping Website

- **Software Architecture:** Organizing the application into User, Product, Order, and Payment modules and choosing a Layered Architecture or Microservices.
- **System Architecture:** Deciding to deploy the application on multiple servers, use a load balancer, PostgreSQL, Redis, and connect to an external payment gateway.

Without Software Architecture & System Design

A system may still work initially, but as it grows, it is likely to suffer from:

- **Poor maintainability** – Code becomes difficult to understand, modify, and debug.
- **Limited scalability** – The system struggles to handle increasing users or traffic.
- **Low performance** – Slow response times and inefficient resource usage.
- **High coupling** – Components become tightly dependent on each other, making changes risky.
- **Reduced reliability** – A failure in one component can affect the entire system.
- **Security weaknesses** – Security concerns are often added as an afterthought instead of being built into the design.
- **Higher development costs** – New features take longer to implement and are more likely to introduce bugs.
- **Expensive redesigns** – Poor early decisions become difficult and costly to reverse later.

02

Core Architectural Patterns

Architectural Patterns

Architectural patterns are reusable, high-level solutions for **organizing the structure** of a software system.

They define how major components are organized and **communicate** to solve common architectural problems.

Let’s solve some problems

Problem #1

Everything is inside one file.

```
app.post("/orders",()=>{
    // validation
    // business
    // SQL
    // response
})
```

What's wrong?

Layered Architecture

Layered Architecture

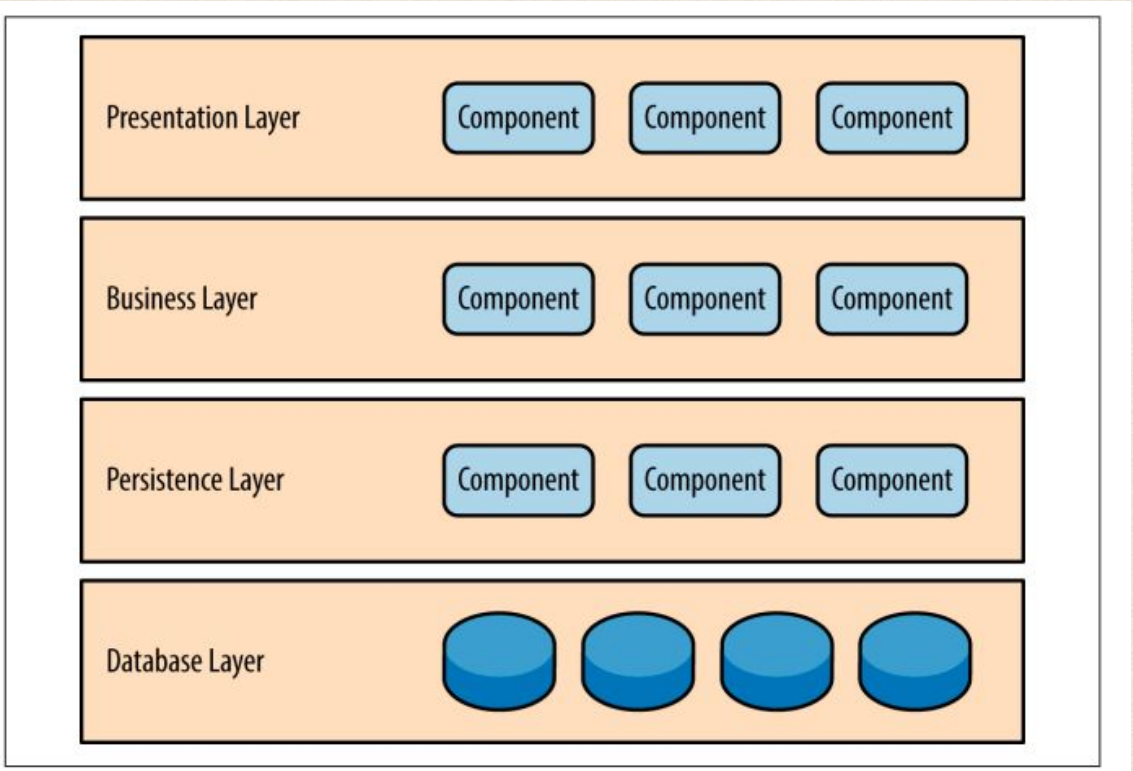


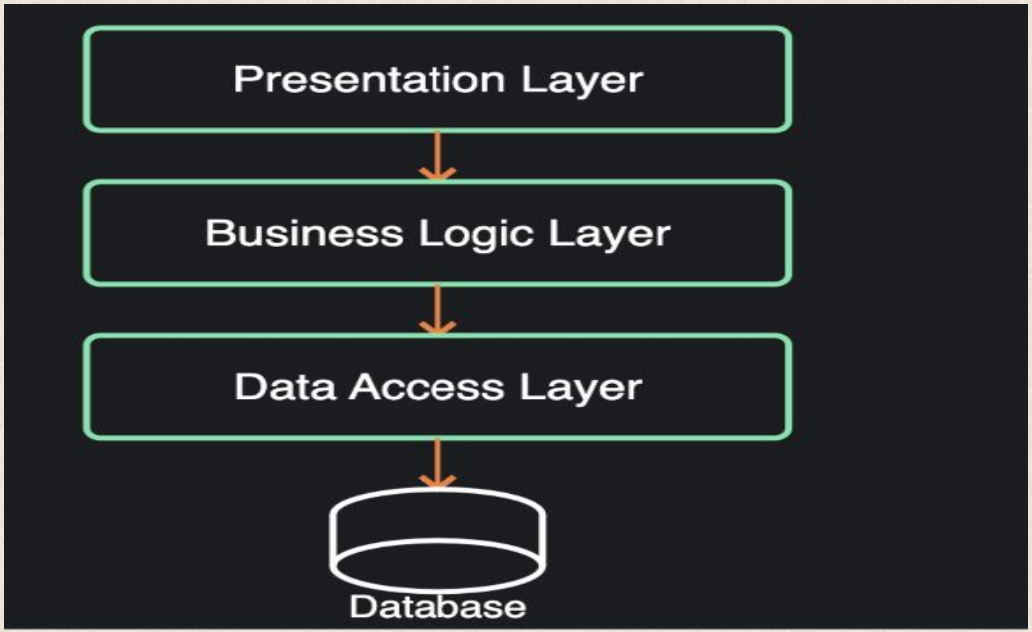
Figure 1-1. Layered architecture pattern

Layered Architecture

Client
↓
Controller
↓
Service
↓
Repository
↓
Database

Layered / N-Tier Architecture

Definition: Organizes an application into **layers**, where each layer has a specific **responsibility** and **communicates** only with adjacent layers



Layered / N-Tier Architecture

Used when: **CRUD applications, enterprise systems, small-to-medium projects.**

Pros: **Easy to understand , clear separation of concerns , easy to maintain , widely supported.**

Cons: **Can become tightly coupled , business logic may leak into other layers , not ideal for highly scalable systems.**

Layered / N-Tier Architecture

1 **Overall agility**

Low

2 **Ease of deployment**

Low

3 **Testability**

High

4 **Performance**

Low

5 **Scalability**

Low

6 **Ease of development**

High

Problem #2

We are replacing Stripe with Paymob.

Should we edit business logic?

Hexagonal Architecture (Ports & Adapters)

Business logic never changes.
Technology changes.

Business Logic

↓

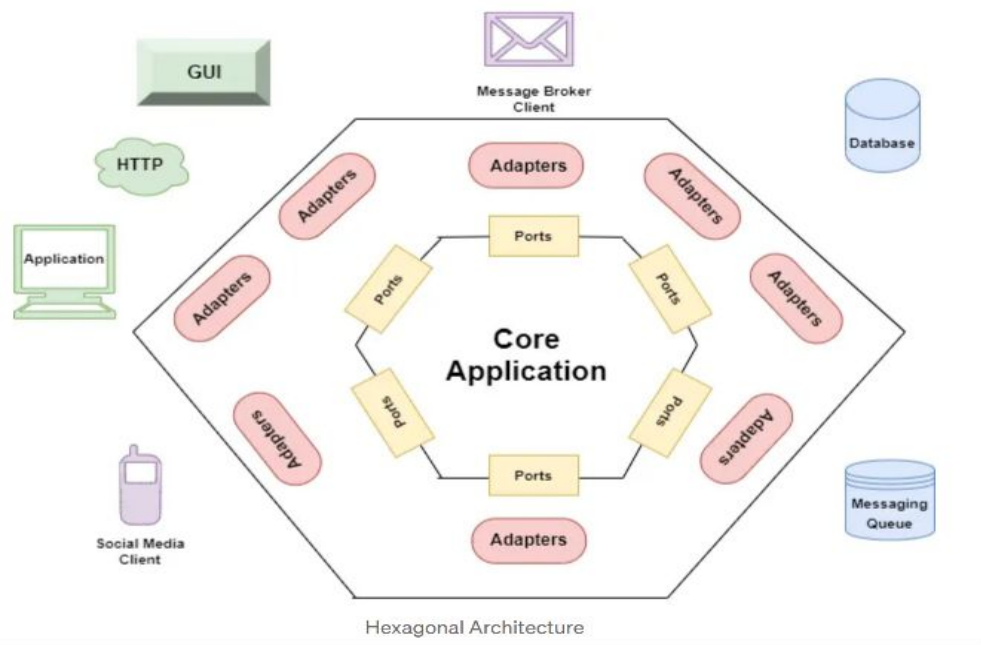
Interface (Port)

↓

Stripe Adapter

Hexagonal Architecture (Ports & Adapters)

Definition: Places **business logic** at the center while isolating external systems (database, APIs, UI) through **interfaces** ("ports") and **adapters** that implement those interfaces.



Hexagonal Architecture

Hexagonal Architecture (Ports & Adapters)

Used when: Complex business logic · testability is important · underlying technologies may change.

Pros:Highly testable · loose coupling · easy to replace external systems · business logic independent of frameworks.

Cons: More abstraction · extra code · overkill for simple CRUD apps.

Hexagonal Architecture (Ports & Adapters)

1 **Overall agility**

High

2 **Ease of deployment**

Medium

3 **Testability**

Very High

4 **Performance**

Medium

5 **Scalability**

Medium

6 **Ease of development**

Low

Problem #3

- The company grows -> 100 developers.
- Question?

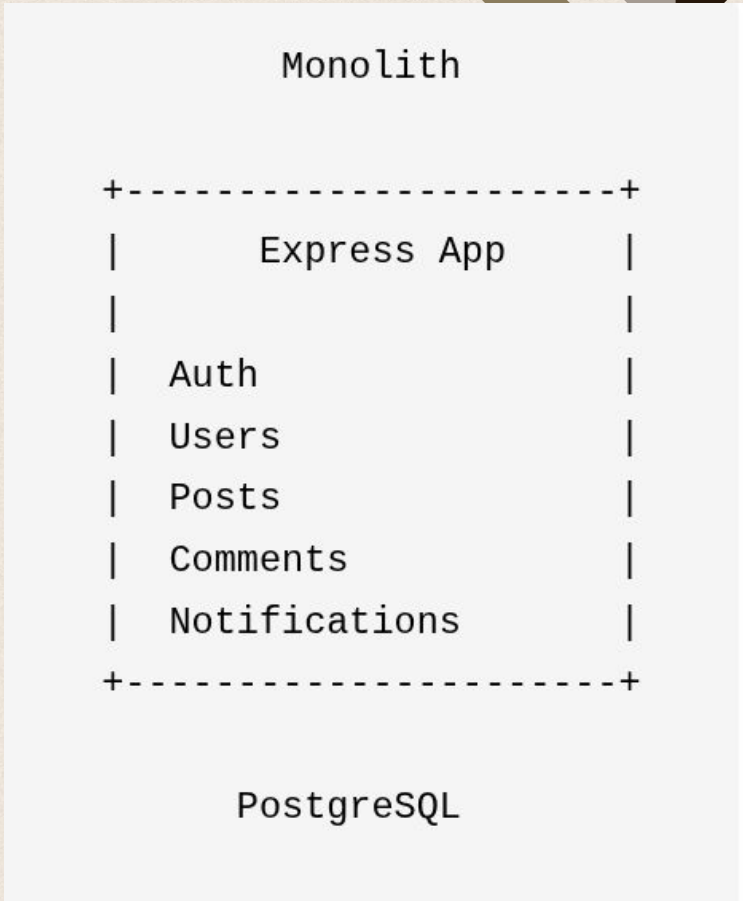
  Should everyone edit one project?

Monolith vs Modular Monolith vs Microservices

Monolith

- What is it?
  - A single application that contains all the features and is deployed as one unit.

- Everything is one application,One deployment,One server



Monolith

Used when:MVP,Startup,Small teams,University projects,Most CRUD applications

Pros:Simple,Easy debugging,Easy deployment,Easy local development,Perfect for startups

Cons:Large codebase,Everyone works on the same project,Deploying one feature deploys everything, Harder to scale teams

Question

If Facebook started today with 3 developers...

Would you choose Microservices?

Question

Five years later.
3 developers -> 120 developers

Now everyone edits the same project.
Merge conflicts.
Slow deployments.
Hard to know who owns what.

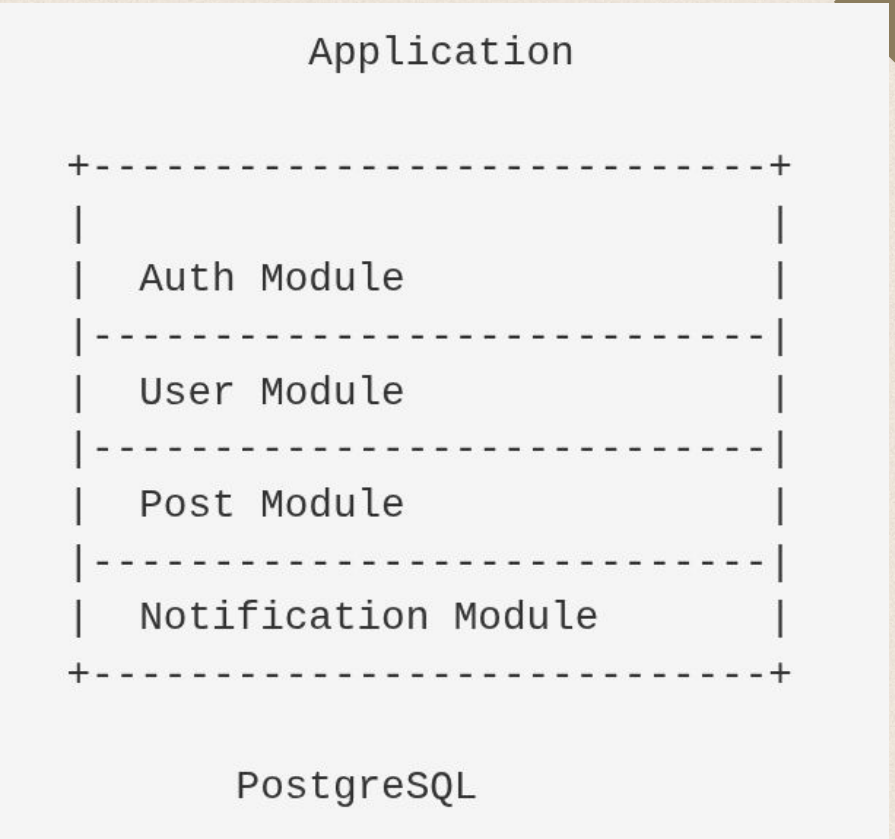
Question

Can we stay one application...

but organize it better?

Modular Monolith

- What is it?
    - One deployment.
    - One application.
    - Multiple independent modules.



Modular Monolith

Pros:One deployment, Clear ownership, Easy onboarding, Easier future migration, Great for growing startups

Cons:One database (usually), Large deployments eventually

Why is this important?

Suppose Later You want Microservices.

Instead of rewriting Everything You extract

Notification Module

↓

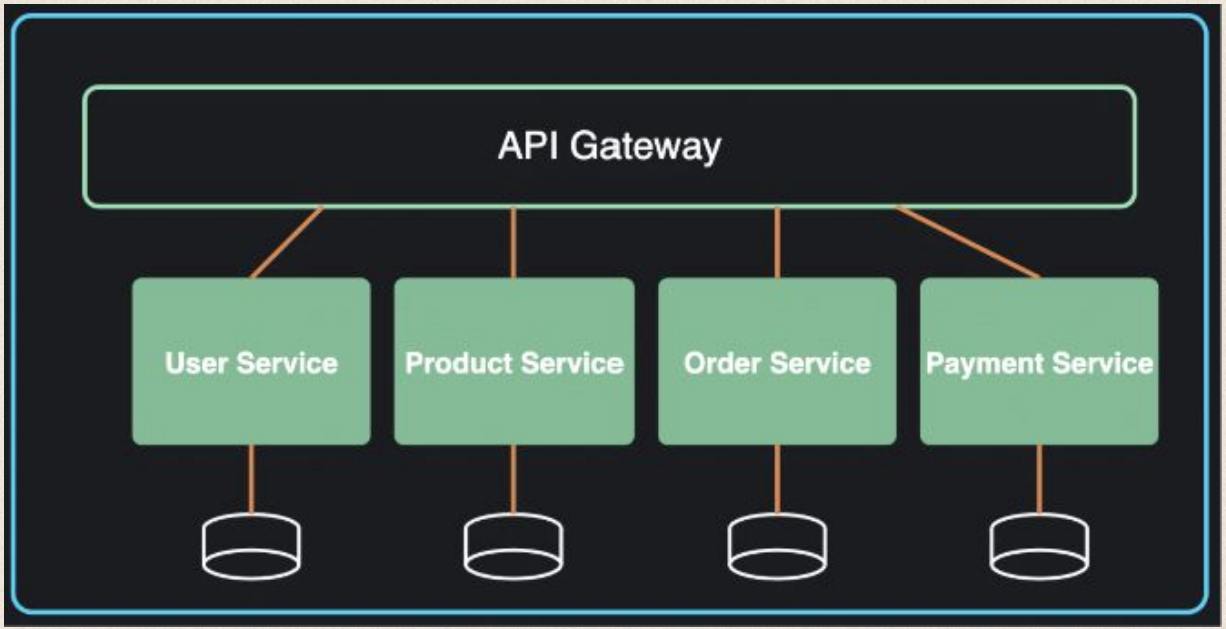
Notification Service

This is why people call Modular Monolith
A stepping stone to Microservices.

	Monolith	Modular Monolith
Deployments	One	One
Code Organization	Loose	Strict Modules
Team Ownership	Difficult	Easier
Easy to Start	True	True
Easy to Grow	False	True
Microservice Ready	False	True

Microservices

Definition: Breaks an application into **small, independent services,** each responsible for one business capability (e.g., User Service, Order Service, Payment Service, Inventory Service), **communicating over the network.**



Microservices

Used when: Large applications · multiple development teams · independent deployment is required.

Pros:Independent deployment · independent scaling · fault isolation · technology flexibility (different services can use different stacks).

Cons: Complex inter-service communication · distributed debugging · higher infrastructure cost · data consistency challenges across services.

Microservices

1 **Overall agility**

High

2 **Ease of deployment**

High

3 **Testability**

High

4 **Performance**

Low

5 **Scalability**

High

6 **Ease of development**

High

Problem #4

Order created Need to call:

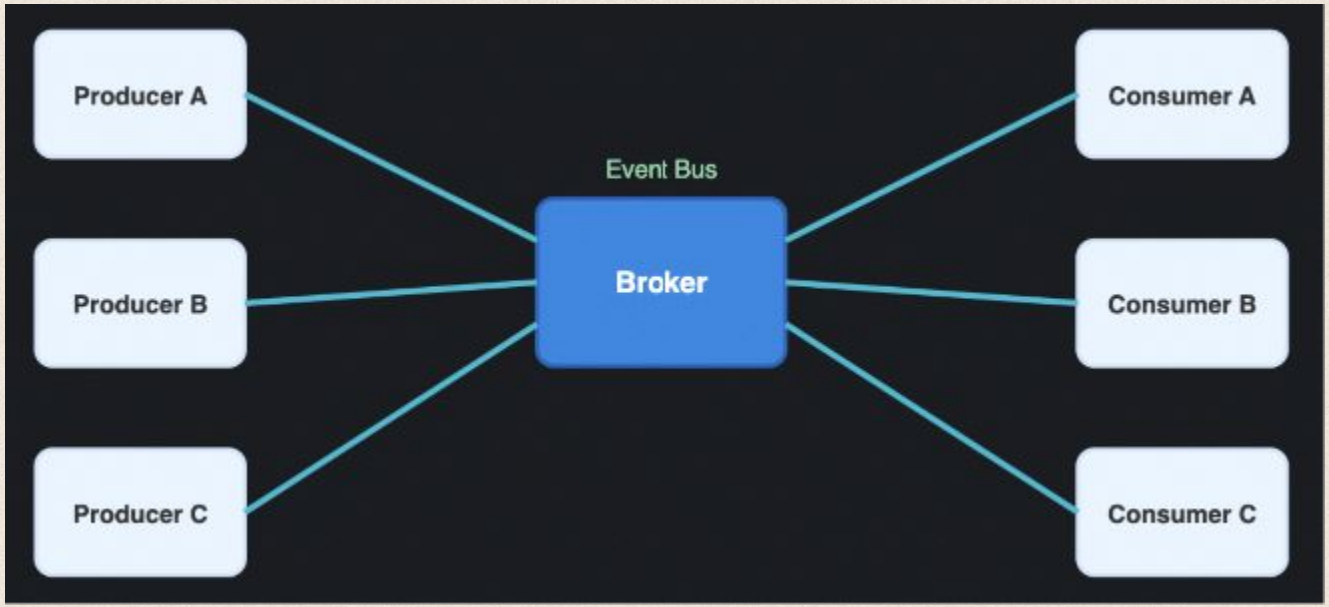
- Email
- Analytics
- Notification
- Inventory

Question

Should Order Service call all of them?

Event-Driven Architecture

Definition: Components **communicate by publishing and consuming events** through a message broker instead of calling each other directly .



Event-Driven Architecture

Used when:Notifications · real-time systems · asynchronous processing.

Pros:Loose coupling · highly scalable · easy to integrate new consumers · handles traffic spikes well.

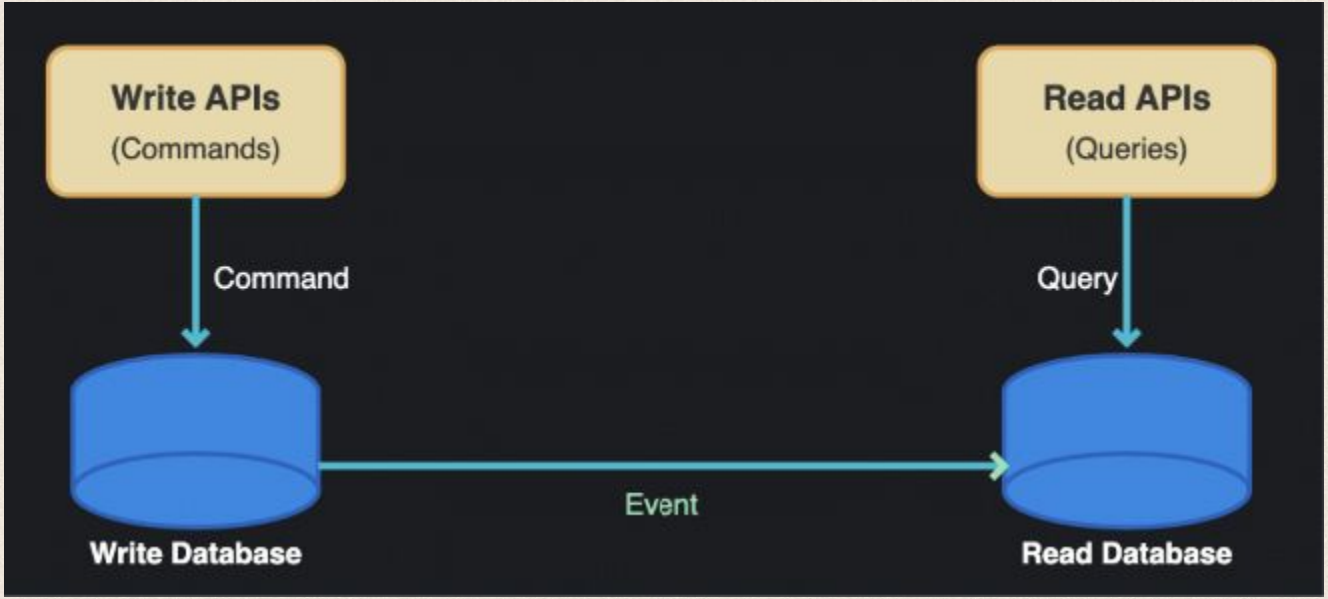
Cons:Hard to debug ("where did this event go") · event ordering issues · eventual consistency · duplicate event handling.

Event-Driven Architecture

1	Overall agility	High
2	Ease of deployment	High
3	Testability	Low
4	Performance	High
5	Scalability	High
6	Ease of development	Low

CQRS (Command Query Responsibility Segregation)

Definition: Separates read operations (Queries) f**rom write operations** (Commands), often with a separate read model and write model.



CQRS (Command Query Responsibility Segregation)

- **Instagram has Millions read, Thousands write.**
- **Should database treat both equally?**
  - **No.**
- **Separate Reads Writes.**

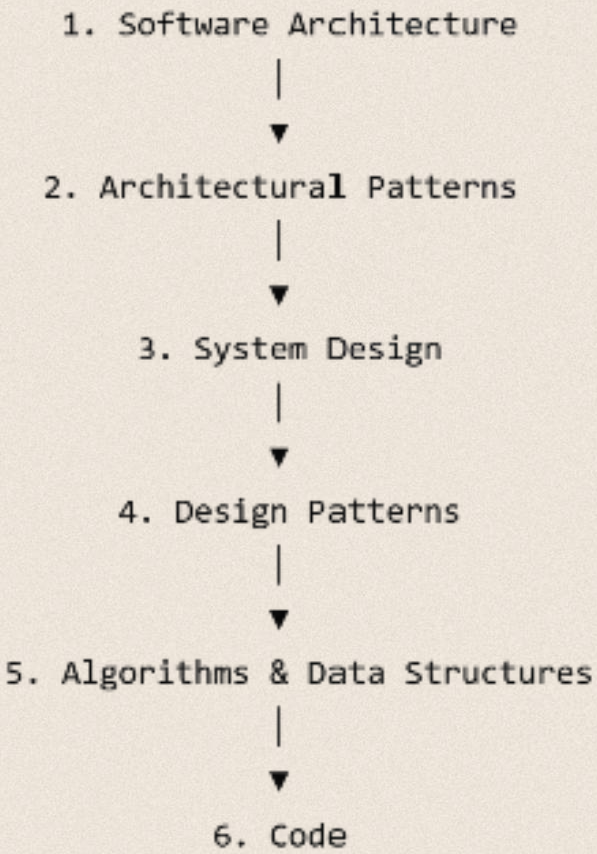
CQRS (Command Query Responsibility Segregation)

Used when:Read-heavy systems · dashboards · analytics.

Pros:Better read performance · independent scaling of reads vs. writes · optimized read/write models.

Cons:More complexity · requires syncing data between models · usually paired with Event Sourcing to be worth the overhead.

Think of them as different levels of abstraction



Pattern	Solves
Layered	Mixed responsibilities
Hexagonal	Business depending on technology
Monolith	Small/simple systems
Modular Monolith	Growing codebase
Microservices	Scaling teams & deployments
Event Driven	Loose communication
CQRS	Read-heavy systems

So far we've been organizing code.

Now let's organize an entire system.

03

System Design Fundamentals

System Design Fundamentals

Functional vs Non-Functional Requirements

- **Functional** — what the system does (e.g., "users can post photos").

- **Non-functional** — how well it does it (e.g., latency under 100ms, 99.99% uptime, 1M requests/sec).
  These constraints usually shape the architecture more than the features do.

The Four Questions Every System Designer Asks

Can it handle more users?

Scalability

Can it survive failures?

Availability

The Four Questions Every System Designer Asks

Can it stay fast?

Performance

Will the data stay correct?

Consistency

How do engineers actually achieve these goals?

System Design provides us with a toolbox.

Each tool improves one or more of these goals.

Scalability

Vertical vs Horizontal Scaling

Vertical Scaling :

- **Bigger Machine**
  - **- More RAM**
  - **- More CPU**
  - **- More Storage**
- **Pros - Very easy**
- **Cons - Expensive - Has limits**

Scalability

Vertical vs Horizontal Scaling

- Horizontal Scaling
- More Servers
  - Server 1
  - Server 2
  - Server 3
- Pros - Almost unlimited - Industry standard
- Cons - More complex

Scalability

Load Balancer

- we have 20 servers.
- But all of them still use ONE database.
- What happens now?
  - Database bottleneck.

- What is load balancer?
- A server that sits before your application and distributes requests.

Availability

- Our server crashes.
- Should Facebook stop working? No.
- We want High Availability.
- Meaning... Even if part of the system fails... Users can still use it.
- How?
  - Three ideas.
    - Redundancy
    - Failover
    - Replication

Availability

Redundancy

- **What?**
  - **Having backups.**
- **One Server -> Three Servers**

Not only servers.

Can be

- **Database**
- **Network**
- **Load Balancer**

Availability

Failover

- What?
  - Automatically switching to another server.
- How?
  - Usually The Load Balancer checks GET /health every few seconds.
    - If unhealthy
      - Remove server.

Availability

Replication

- What?
  - Copying data.
- Why?
  - If one replica dies
  - Read from another.

Performance

- How long would YOU wait before closing Instagram?
- 1 second? 3 seconds? 10 seconds?
- Performance means
  - Fast Responses (Low Latency)
  - Handling many requests (High Throughput)

Performance

Cache

- What?
- Temporary fast storage.
- Instead of
  - User -> Database
- Use
  - User -> Redis -> Database

Consistency

- What?
  - Making sure everyone sees the correct data.
- How?
  - Transactions
  - Locks
  - Consensus

Consistency

- Imagine a Bank.
- Balance $1000 You transfer $1000
- Should another ATM still show $1000? NO.
- Banking requires strong consistency.

- Instagram Likes 500 501 500 502
- Nobody notices.
- Eventually Consistent is okay.

Activity!

Design Facebook:

- Login
- Create Post
- Like Post
- Comment
- Notifications

Your turn

Discord:

- Chat
- Voice
- Notifications
- Friends
- Online Status

Thanks!